Certain Informations

1. **19th Feb (Thursday) will be your Mid term**. All the topics till 6th week will be included.
2. **Missed lab need to be repeated by taking permission from the lab-inchrage(ME)** on a letter and the letter need to submitted to the exchaged lab teacher, who will sign are return the letter. This letter has to be submitted ME.
3. If anyone is missing any regular labs, they have to complete their **repeatition lab max of one week time**, later isn't considered for evaluation.
4. If and only if the missed labs are **repeated with other batchs at the college, then only the week will be considered for evaluation**.
5. Those who miss regular, kindly **message me on TEAMS as proof**. My email id is: **vidyarao.mit@manipal.edu**. **DO NOT EMAIL ME.**

Data Visualization Week-5

- BTech Computer Science Stream , Februry 2026
- Week 5 - Data Cleaning and Preparation
- Name: Vidya Rao
- Reg Number: 12345678
- Date: 05/02/2026

Goal

In this session, we learn a **general workflow** for cleaning tabular data:

1. Load data (CSV → DataFrame)
2. Inspect structure and summary
3. Detect missing values (including placeholders like `-1`)
4. Replace missing values (mean strategy)
5. Detect noisy values / outliers
6. Replace noise values (median strategy)
7. Validate the changes and export results

Import Required Libraries

In [3]:
```
import pandas as pd
import numpy as np

pd.set_option("display.max_columns", None)     #displays all the colunms.

pd.set_option("display.width", 1000)           #increases the screen width so tha
                                               #displayed in a single line withou
```

```
print("Environment ready")
```

```
Environment ready
```

Display Settings in Pandas (For Better Readability)

- display.max_columns = None
  - Shows all columns of the DataFrame without hiding any.
- display.width = 1000
  - Increases the screen width so that tables are displayed in a single line without wrapping.

Step 1: Create a CSV File (Demo Dataset)

- In labs and exams, data is usually provided as a **CSV file**.
- We first simulate that situation by creating and saving a CSV file.

In [4]:
```
demo_data = pd.DataFrame({
    "device_id": ["D1", "D2", "D3", "D4", "D5"],
    "temperature": [28, -1, 30, 95, 29],   # -1 → missing, 95 → noisy
    "humidity": [65, 70, -1, 68, 20],       # -1 → missing, 20 → noisy
    "pressure": [1012, 1013, -1, 1011, 500] # -1 → missing, 500 → noisy
})

demo_data
```

Out[4]:

	device_id	temperature	humidity	pressure
0	D1	28	65	1012
1	D2	-1	70	1013
2	D3	30	-1	-1
3	D4	95	68	1011
4	D5	29	20	500

Write to CSV using to_csv()

In [5]:
```
demo_data.to_csv("Demo_SensorData.csv", index=False)
print("Demo_SensorData.csv created")
```

```
Demo_SensorData.csv created
```

Step 2: Load Data (CSV → DataFrame)

All processing must begin by **reading the CSV file**.

In [6]:
```
df = pd.read_csv("Demo_SensorData.csv")
df
```

Out[6]:

	device_id	temperature	humidity	pressure
0	D1	28	65	1012
1	D2	-1	70	1013
2	D3	30	-1	-1
3	D4	95	68	1011
4	D5	29	20	500

Step 3: Inspect the Dataset

Before cleaning, always inspect:

- Shape (rows × columns)
- Data types
- Basic statistics

In [7]:
```
df.shape     #Shape (rows × columns)
```

Out[7]: (5, 4)

In [8]:
```
df.info()    #Data types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column       Non-Null Count  Dtype 
---  ------       --------------  ----- 
 0   device_id    5 non-null      object
 1   temperature  5 non-null      int64 
 2   humidity     5 non-null      int64 
 3   pressure     5 non-null      int64 
dtypes: int64(3), object(1)
memory usage: 292.0+ bytes
```

In [9]:
```
df.describe()    #Basic statistics
```

Out[9]:

	temperature	humidity	pressure
count	5.000000	5.000000	5.000000
mean	36.200000	44.400000	707.000000
std	35.351096	32.761258	453.649093
min	-1.000000	-1.000000	-1.000000
25%	28.000000	20.000000	500.000000
50%	29.000000	65.000000	1011.000000
75%	30.000000	68.000000	1012.000000
max	95.000000	70.000000	1013.000000

5-Number Summary

The 5-number summary includes:

- Minimum
- 25th percentile (Q1)
- Median
- 75th percentile (Q3)
- Maximum

This helps identify missing and noisy values.

Step 2b: 5-number summary (min, Q1, median, Q3, max)

What are we doing?

We compute the 5-number summary for numeric columns.

Why are we doing it?

Week 5 requires 5-number summary. It helps detect:

- missing values (-1)
- noisy/outlier values (very high/low)

Outcome / What to expect

A table with 5 rows: 0%, 25%, 50%, 75%, 100%

Notice:

- If 0% (min) is -1 → missing placeholders exist.
- If 100% (max) is very large → likely noise exists.

In [10]:
```
numeric_cols = ["temperature", "humidity", "pressure"]

df[numeric_cols].quantile([0, 0.25, 0.5, 0.75, 1.0])
```

Out[10]:

	temperature	humidity	pressure
0.00	-1.0	-1.0	-1.0
0.25	28.0	20.0	500.0
0.50	29.0	65.0	1011.0
0.75	30.0	68.0	1012.0
1.00	95.0	70.0	1013.0

STOP & TYPE (Students)

Task

1. Print the maximum value of each numeric column.
2. Print the median of each numeric column.

In [11]:
```
df[numeric_cols].max()
```

Out[11]:
```
temperature      95
humidity         70
pressure       1013
dtype: int64
```

In [12]:
```
df[numeric_cols].median()
```

Out[12]:
```
temperature      29.0
humidity         65.0
pressure       1011.0
dtype: float64
```

Step 4: Detect Missing Values

In this dataset, missing values are represented using `-1`.

Why are we doing it?

Because `-1` is not a real reading here. It represents missing data. We must fix missing values before handling noise.

In [13]:
```
(df[numeric_cols] == -1).sum()
```

Out[13]:
```
temperature    1
humidity       1
pressure       1
dtype: int64
```

Step 5: Replace Missing Values Using Mean

Procedure:

1. Ignore `-1`
2. Compute mean of remaining values
3. Replace `-1` with the mean

Why are we doing it?

- Missing placeholders distort summaries and calculations.
- Mean replacement is a beginner-friendly approach.

In [14]:
```
df_mean_cleaned = df.copy()

for col in numeric_cols:
    mean_value = df_mean_cleaned.loc[df_mean_cleaned[col] != -1, col].mean()
    df_mean_cleaned[col] = df_mean_cleaned[col].replace(-1, mean_value)

df_mean_cleaned
```

Out[14]:

	device_id	temperature	humidity	pressure
0	D1	28.0	65.00	1012
1	D2	45.5	70.00	1013
2	D3	30.0	55.75	884
3	D4	95.0	68.00	1011
4	D5	29.0	20.00	500

Validate Missing Value Handling

We verify:

1. No -1 values remain
2. 5-number summary now makes sense (min is no longer -1)

Why are we doing it?

- Cleaning is incomplete unless you verify it.

In [20]:
```
print("Missing placeholder counts after mean replacement:")
print((df_mean_cleaned[numeric_cols] == -1).sum())

print("\n5-number summary after missing treatment:")
df_mean_cleaned[numeric_cols].quantile([0, 0.25, 0.5, 0.75, 1.0])
```

```
Missing placeholder counts after mean replacement:
temperature    0
humidity       0
pressure       0
dtype: int64

5-number summary after missing treatment:
```

Out[20]:

	temperature	humidity	pressure
0.00	28.0	20.00	500.0
0.25	29.0	55.75	884.0
0.50	30.0	65.00	1011.0
0.75	45.5	68.00	1012.0
1.00	95.0	70.00	1013.0

Step 6: Detect Noisy Values

What are we doing?

- We recognize that some values are abnormally large (noise).
- Noisy values are **extremely large or small** values that distort analysis.

Why are we doing it?

- Noise distorts analysis and statistics.
- Mean is very sensitive to noise.
- Median is more stable.

Outcome

In our dataset:

- temperature 95 looks too high
- pressure 500 looks too low

Step 7: Replace Noisy Values Using Median

What are we doing?

For each numeric column:

- compute the median
- replace values that are "too large" using a simple threshold: `value > 3 × median`

Why are we doing it?

Median is robust (not strongly affected by extreme values).

Outcome

- extreme high values should reduce
- dataset should look more realistic after replacement

In [17]:
```
df_final = df_mean_cleaned.copy()

for col in numeric_cols:
    median_value = df_final[col].median()
    df_final.loc[df_final[col] > median_value * 3, col] = median_value

df_final
```

Out[17]:

	device_id	temperature	humidity	pressure
0	D1	28.0	65.00	1012
1	D2	45.5	70.00	1013
2	D3	30.0	55.75	884
3	D4	30.0	68.00	1011
4	D5	29.0	20.00	500

Step 8: Validate Cleaning

What are we doing?

We compare 5-number summaries:

- Before cleaning
- After missing-value fix
- After noise fix

Why are we doing it?

This helps answer the lab-style question: "Was the strategy effective?"

Outcome

- Before: min = -1 (missing), max very high (noise)
- After missing fix: no -1
- After noise fix: extreme max reduces

In [21]:
```
print("Before cleaning (treat -1 as NaN for summary):")
display(df[numeric_cols].replace(-1, np.nan).quantile([0,0.25,0.5,0.75,1.0]))

print("\nAfter missing value handling:")
display(df_mean_cleaned[numeric_cols].quantile([0,0.25,0.5,0.75,1.0]))

print("\nAfter noise handling:")
display(df_final[numeric_cols].quantile([0,0.25,0.5,0.75,1.0]))
```

```
Before cleaning (treat -1 as NaN for summary):
```

	temperature	humidity	pressure
0.00	28.00	20.00	500.00
0.25	28.75	53.75	883.25
0.50	29.50	66.50	1011.50
0.75	46.25	68.50	1012.25
1.00	95.00	70.00	1013.00

```
After missing value handling:
```

	temperature	humidity	pressure
0.00	28.0	20.00	500.0
0.25	29.0	55.75	884.0
0.50	30.0	65.00	1011.0
0.75	45.5	68.00	1012.0
1.00	95.0	70.00	1013.0

```
After noise handling:
```

	temperature	humidity	pressure
0.00	28.0	20.00	500.0
0.25	29.0	55.75	884.0
0.50	30.0	65.00	1011.0
0.75	30.0	68.00	1012.0
1.00	45.5	70.00	1013.0

Step 9: Export Cleaned Dataset

What are we doing?

We save the cleaned data into a new CSV file.

Why are we doing it?

Lab tasks and real projects often require:

- saving cleaned data
- using cleaned data later for analysis or models

Outcome

A new file should be created: `Demo_SensorData_Cleaned.csv`

In [19]:
```
df_final.to_csv("Demo_SensorData_Cleaned.csv", index=False)
print("Cleaned CSV file saved successfully")
```

```
Cleaned CSV file saved successfully
```

What students do next (Lab Manual)

Now apply the SAME workflow to the lab dataset:

CSV → Inspect → Missing(-1) → Mean → Noise → Median → Validate → Export

If you follow these steps in this order, you can solve the Week 5 lab exercises independently.

Apply the SAME steps:

1. Load the given CSV file
2. Inspect structure and summary
3. Detect missing values
4. Replace missing values using mean
5. Detect noisy values
6. Replace noisy values using median
7. Validate and export results